

2026-05-17 - Zephyr Project Structure and Build System

Goal

Understand how a Zephyr application is organized and how the build system compiles it.

What you will learn

- The role of `CMakeLists.txt` and `prj.conf`
- How West manages Zephyr projects and sample repositories
- The layout of a typical Zephyr application
- How to build and run a sample application

Overview

A Zephyr application consists of source files, build metadata, and configuration files. The Zephyr build system uses CMake to generate build targets and West as a project manager.

Typical files

- `CMakeLists.txt`: defines the application build, sources, and dependencies
- `prj.conf`: selects Zephyr features with Kconfig symbols
- `src/main.c`: application entry point
- `boards/` or board-specific overlays: optional hardware configuration

Build flow

1. `west init` / `west update` to fetch Zephyr and modules
2. `west build -b <board> <app-dir>` to configure and compile
3. `west flash` or `west debug` to deploy and run

Why this matters

Knowing the build structure helps you create valid Zephyr apps and troubleshoot build failures.

Practice tasks

1. Create a minimal Zephyr app directory with `CMakeLists.txt`, `prj.conf`, and `src/main.c`.
2. Build a sample for a supported board such as `native_posix` or your target board.
3. Open the generated `build/zephyr/zephyr.elf` or build log to inspect compile steps.

Next topic

Tomorrow, we will cover Device Tree basics and how Zephyr maps hardware in software.